

ISTN7MP

Mobile and Personal Information Systems Honours Module – Semester 2 - 2022

Prof Prabhakar Rontala Subramaniam

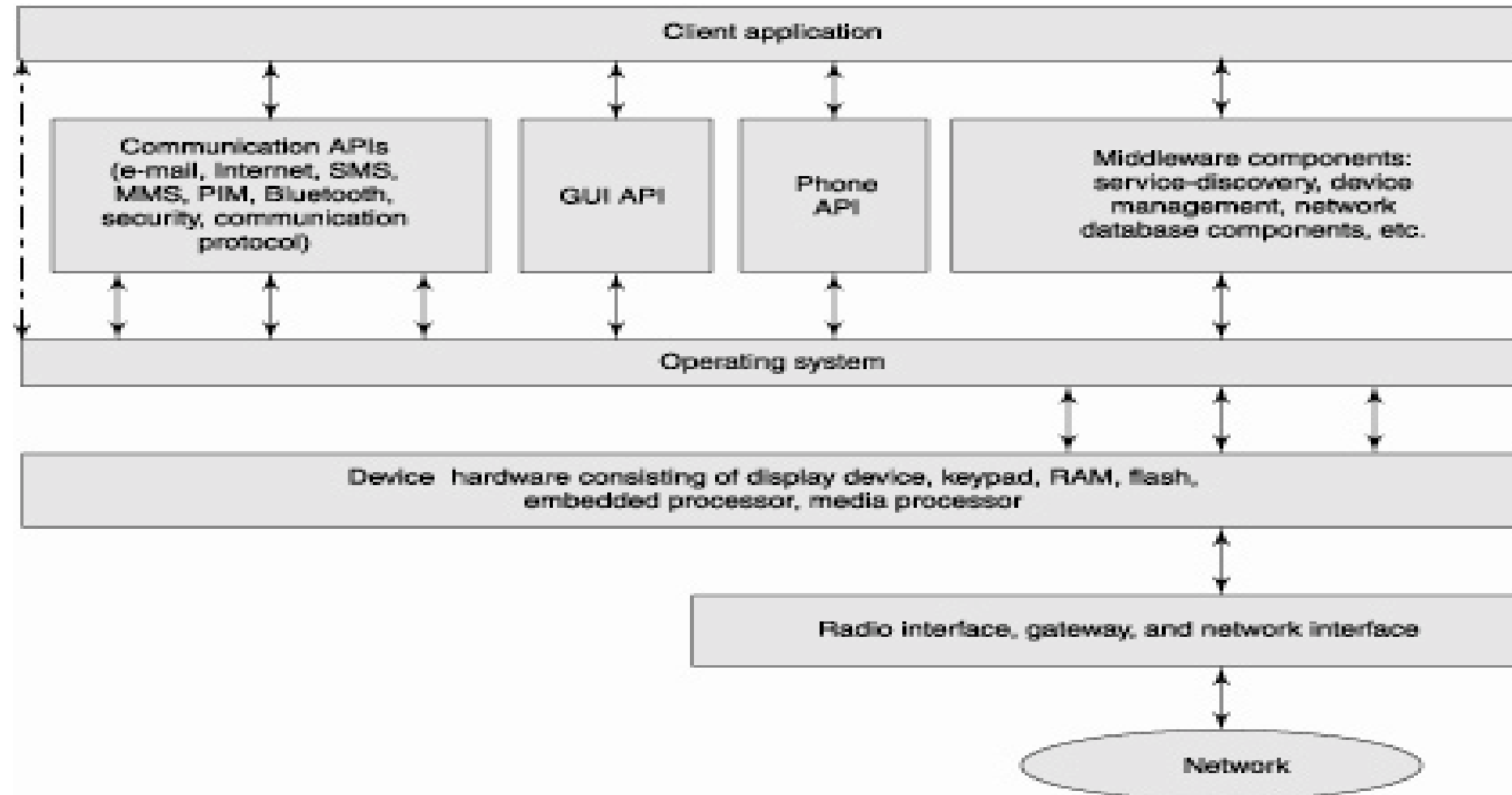
Prof Irene Govender

Mobile OS Architectures

A Discussion

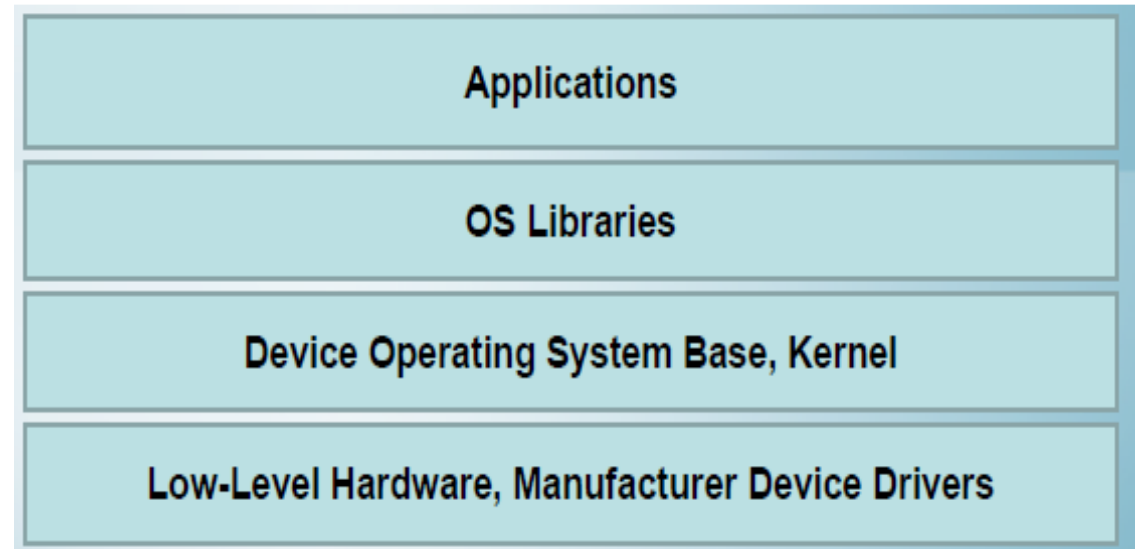
Week 7

Mobile Computing Architectural Layers

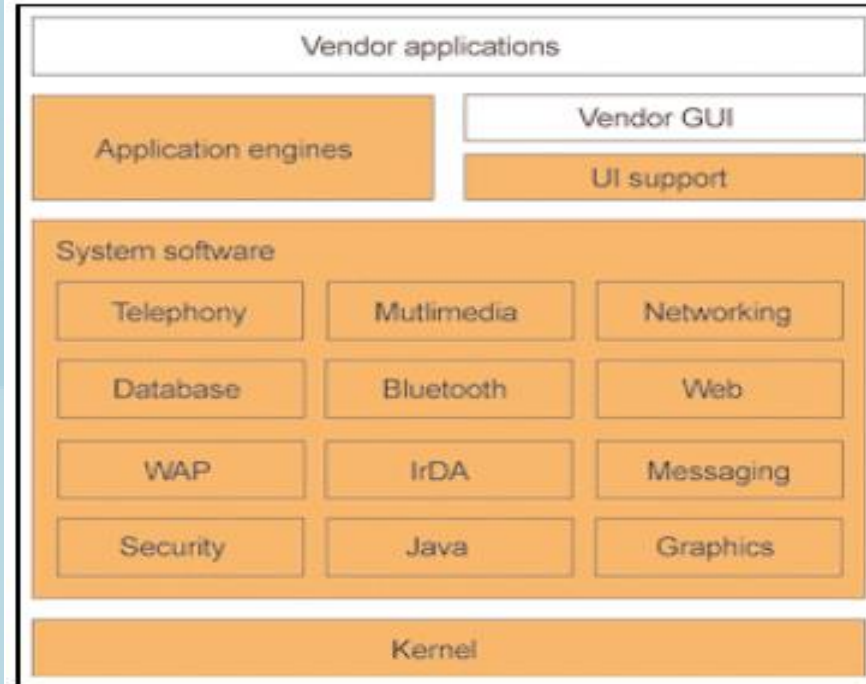
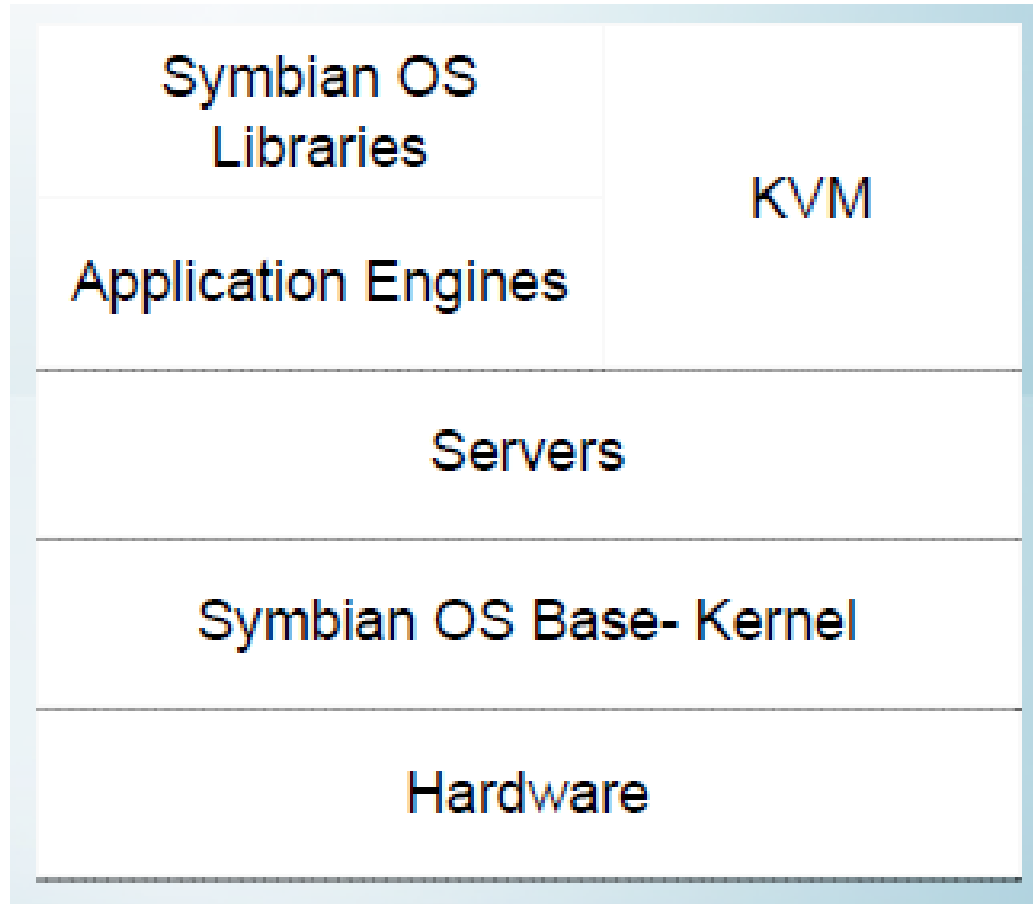


Mobile OS General Structure

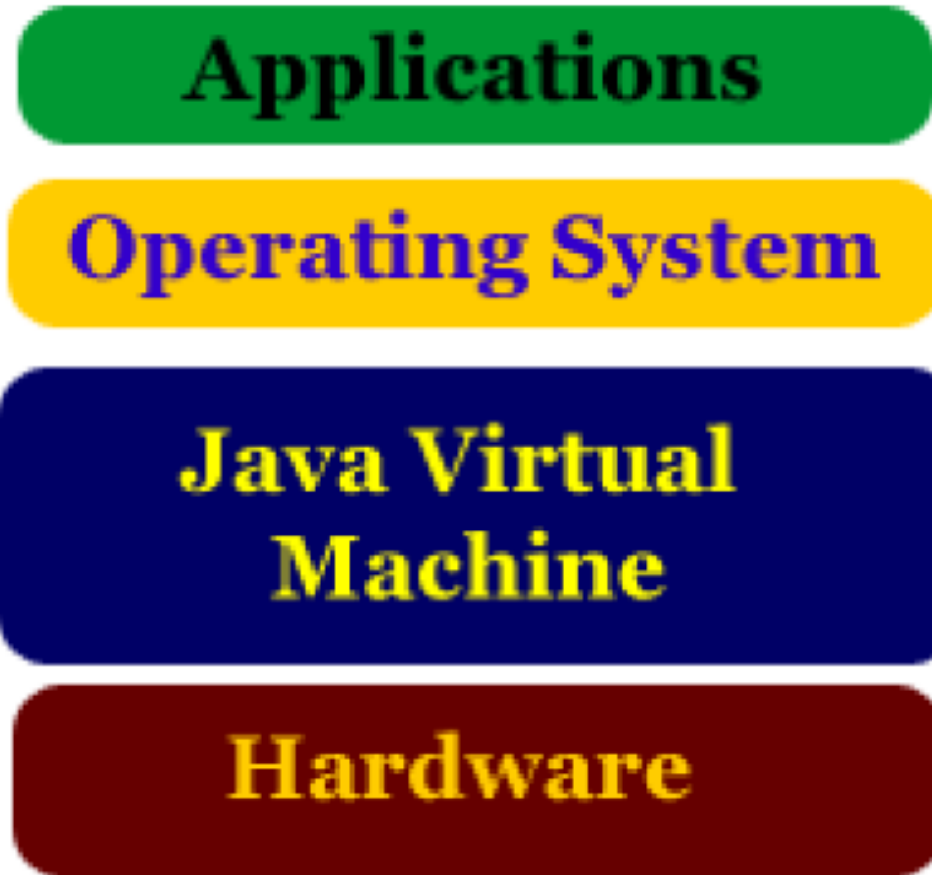
- Mobile OS is a software on top of which other programs called application programs can run on mobile devices like PDAs, Smart Phones & TVs, Tabs, Laptops, Glass, etc.



Symbian Architecture

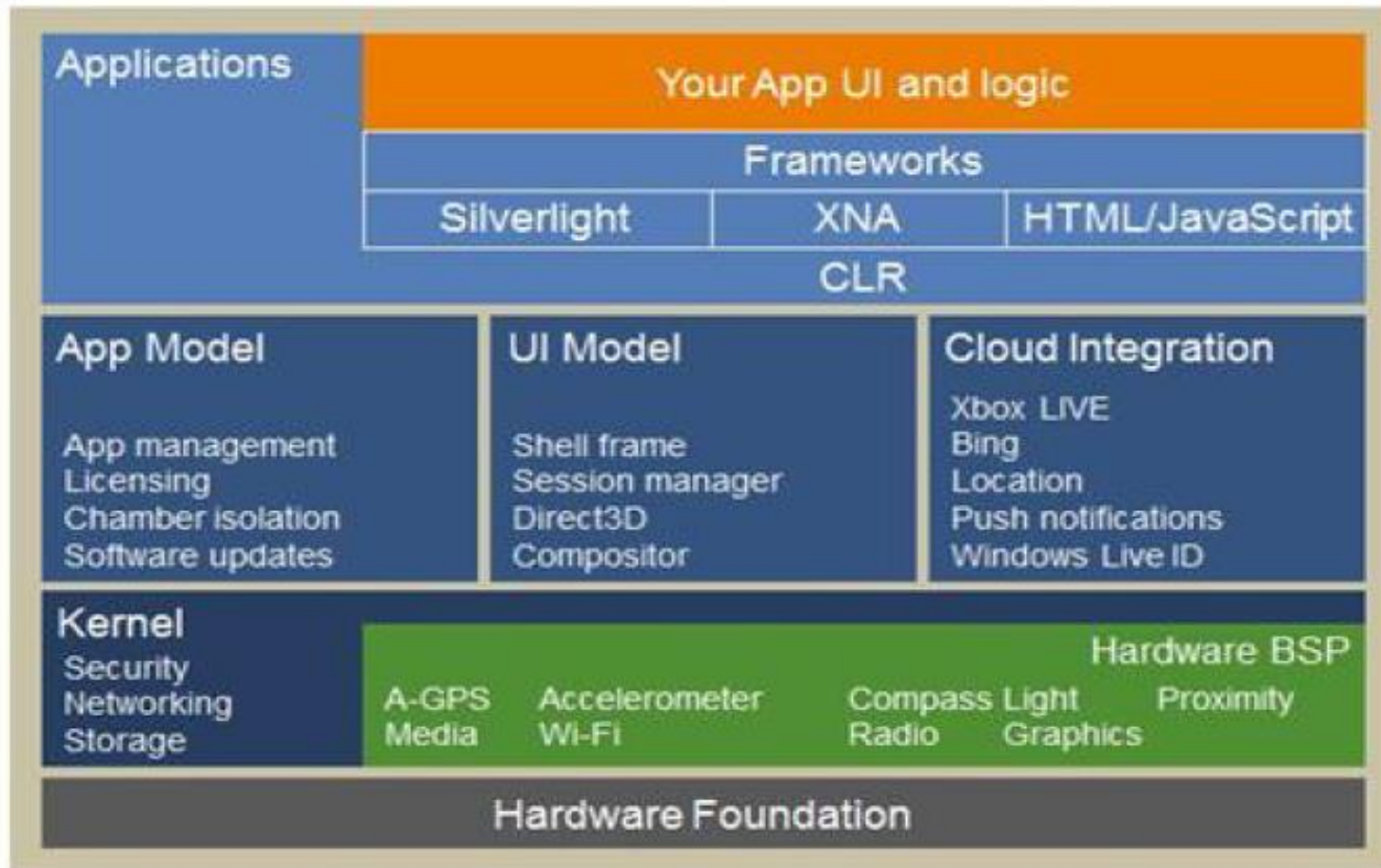


Blackberry OS Architecture

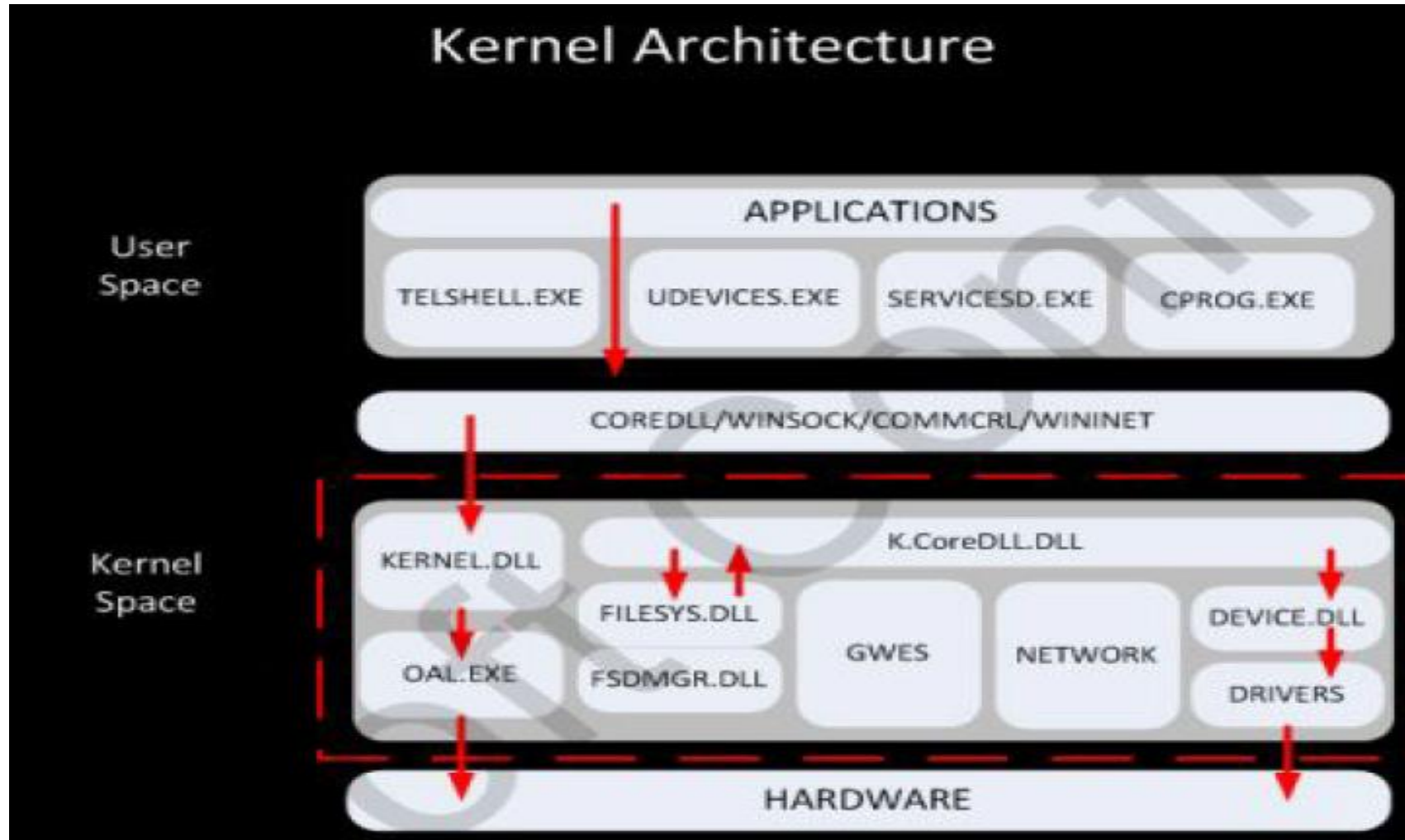


Windows Architecture

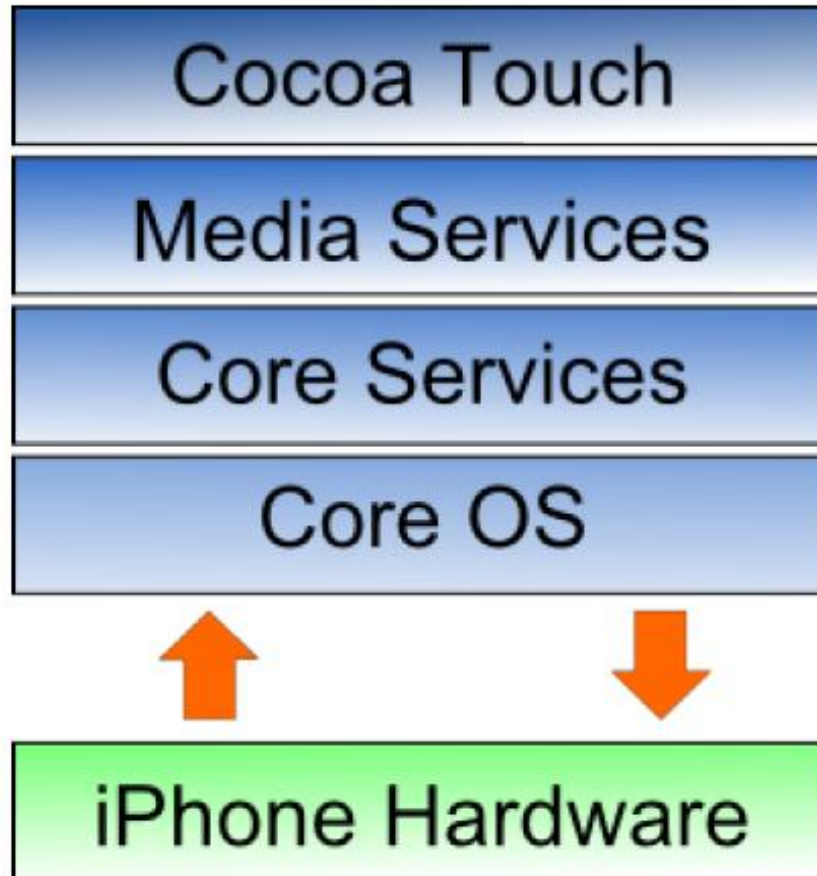
Software Architecture



Windows Architecture (Component Services)



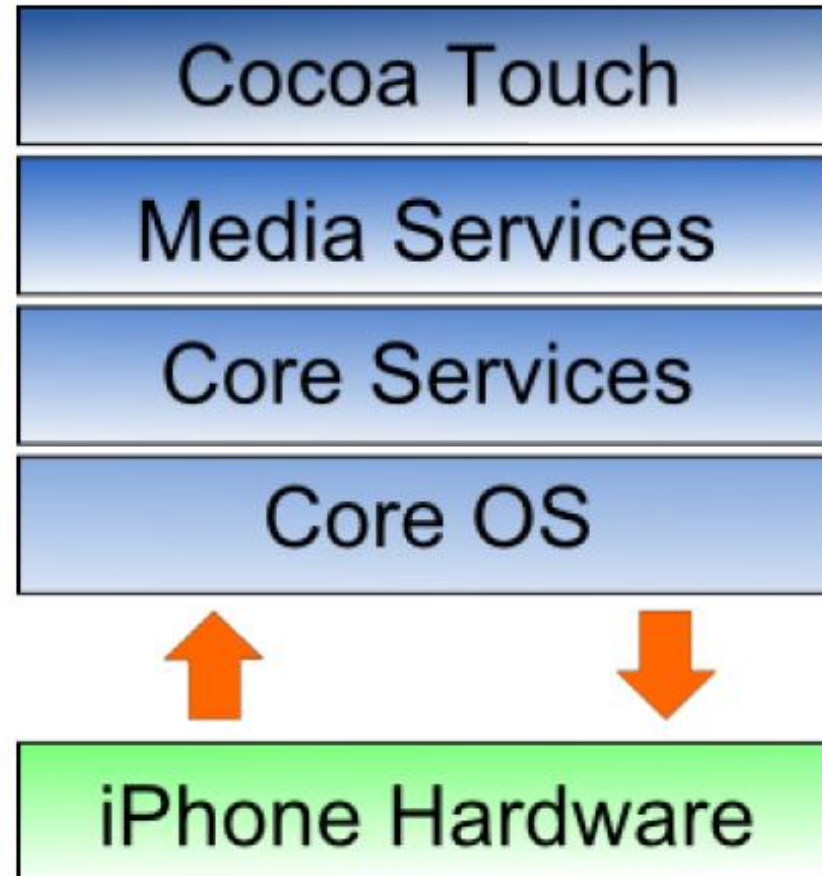
Apple iOS Architecture



History of iOS

- The version history of iPhone OS began with the release of the iPhone on June 29, 2007
- This operating system from Apple Inc. did not have an official name until the release of the iPhone SDK on March 6, 2008. Before then, Apple marketing literature simply stated that the iPhone runs "OS X", a reference to iPhone OS's parent, Mac OS X
- The Developer Beta for version 3.0 was made available on March 17, 2009; iPhone OS 3.0 was released June 17, 2009
- Apple's iOS use Objective-C as the development platform/environment for its applications

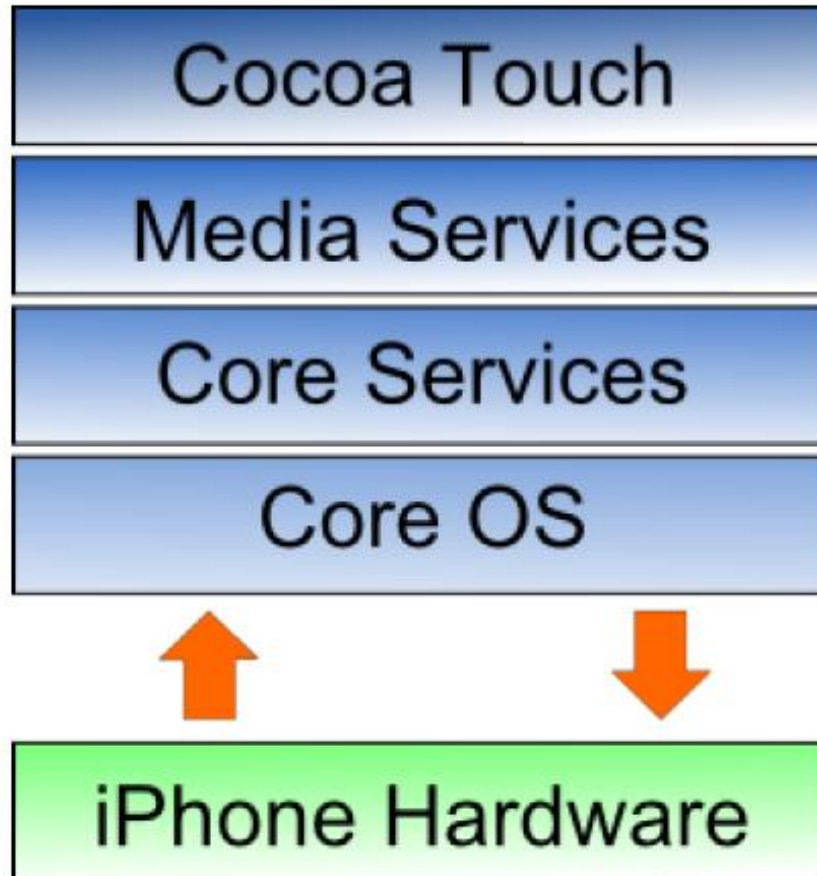
Apple iOS Architecture



Core OS and Core Services Layers

- The Core OS and Core Services layers contain the fundamental interfaces for iPhone OS, including those used for accessing files, low-level data types, Bonjour services, network sockets, and so on.
- These interfaces are mostly C-based and include technologies such as Core Foundation, CF Network, SQLite, and access to POSIX threads and UNIX sockets among others.

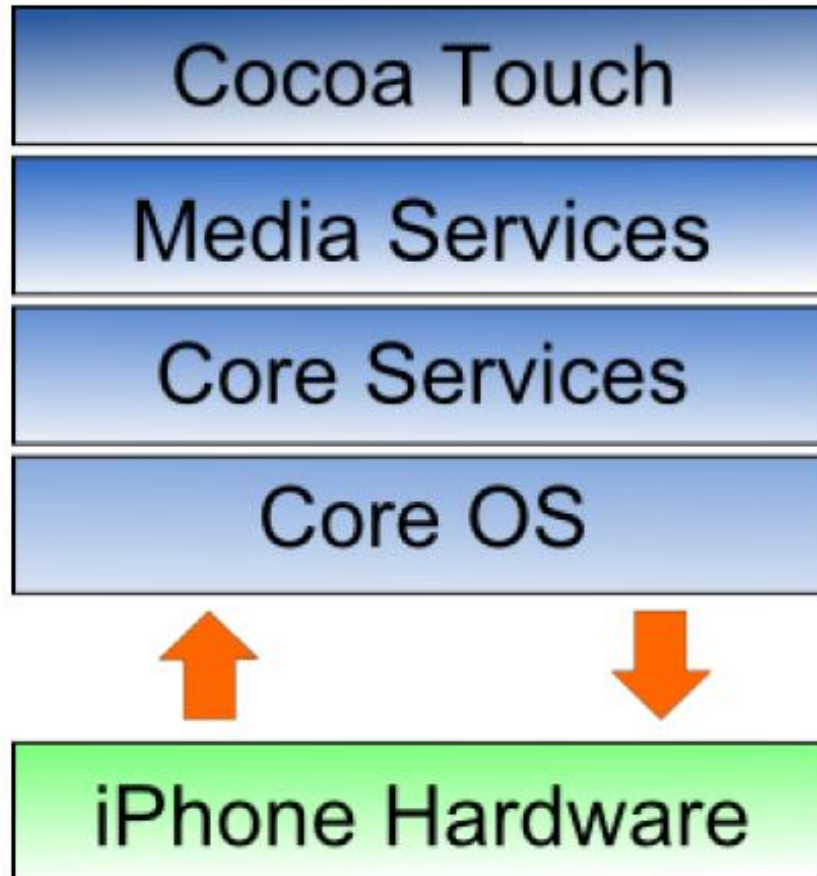
Apple iOS Architecture



Media Layer

- The Media layer contains the fundamental technologies used to support 2D and 3D drawing, audio, and video.
- This layer includes the C-based technologies OpenGL ES (OpenGL for embedded systems), Quartz, and Core Audio.
- It also contains Core Animation, which is an advanced Objective-C based animation engine.
- It uses a mixture of C-based and Objective-C based interfaces.

Apple iOS Architecture



Cocoa Touch Layer

- The Cocoa Touch is a UI framework for building software applications that run on Apple devices (iPhone, iPad, iPod, etc.)
- Apart from being used for iOS it is also used for watchOS (Apple watches) and tvOS (Apple TVs).
- The Cocoa Touch layer provides the fundamental infrastructure used by the applications.
- For example, the Foundation framework provides object oriented support for collections, file management, network operations, and more.
- It is based on Objective-C.

Bonjour Services

Bonjour is a suite of protocols for zero-configuration networking over IP that Apple has submitted to the IETF as part of the ongoing standards-creation process. This section describes the problems that Bonjour solves and how it solves them.

Bonjour is Apple's implementation of a suite of zero-configuration networking protocols. Bonjour is designed to make network configuration easier for users.

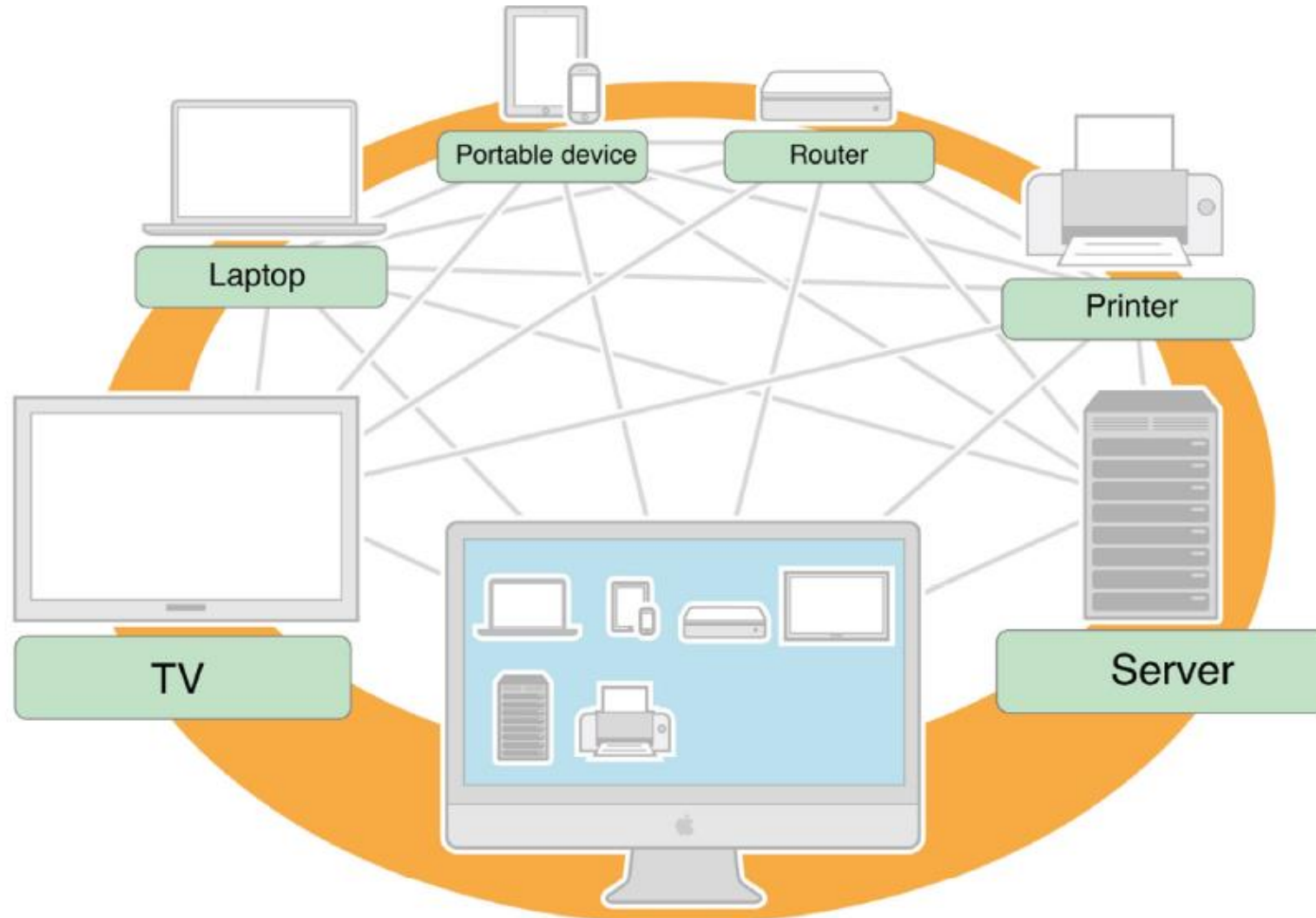
For example, Bonjour lets you connect a printer to your network without the need to assign it a specific IP address or manually enter that address into each computer. With zero-configuration networking, nearby computers can discover its existence and automatically determine the printer's IP address. And if that address is a dynamically assigned address that changes, they can automatically discover the new address in the future.

Apps can also leverage Bonjour to automatically detect other instances of the app (or other services) on the network. For example, two users running an iOS photo sharing app could share photos over a Bluetooth personal area network without the need to manually configure IP addresses on either device.

Zero-Configuration: An Example

- Consider every day task of printing
- Once a printer is configured on the computer or iOS device, its just a matter of choosing the application's print command.
- When the same device is taken to a client's company or a neighbor's house, and if the printer supports Bonjour protocol, printing is just as easy as it was on your local network.
- The device automatically discovers any available printers.

Bonjour Services



Why Bonjour

- The transition from platform-specific protocols (such as AppleTalk, IPX, and NetBIOS) to Internet Protocol (IP).
- The majority of computers and other network devices all use TCP/IP for communication.
- In this transition, the missing functionality is the ability to add devices to a local network and then connect those devices from computers and other devices on the network, all with little or no configuration.
- To support this, a simple and reliable way to configure and browse for services over IP networks is required. This is exactly what Bonjour provides.

Bonjour Solutions

- Bonjour is based on single protocol usage – IP
- Network users no longer have to assign IP addresses, assign host names, or even type in names to access services on the network. Just simply see what network services are available and choose from the list.
- Bonjour provide solution for the following three areas:
 - Addressing (allocating addresses to hosts)
 - Naming (using names to refer to hosts instead of IP addresses)
 - Service Discovery (finding services on the network automatically)

Bonjour Solutions - Addressing

- Addressing problem is solved by self-assigned link-local addressing
- Link-local addressing uses a range of addresses for the local network, typically a small LAN or a LAN segment as specified using IPv6
- Any user or service on a computer or iOS device that supports link-local addressing benefits from the feature automatically. When the host computer encounters a local network, it finds an unused local address and adopts it. No action is required for configuring.

Bonjour Solutions - Naming

- The proposed solution for name-to-address translation on local network uses Multicast Domain Name System (mDNS), in which DNS-format queries are sent over the local network using IP multicast.
- DNS is a internet service that translates domain names into IP addresses
- Bonjour includes a responder to handle mDNS queries for any network service on the host computer or iOS device.
- Bonjour also provides built-in support for Network Address Translation port mapping protocol (NAT-PMP)
- For name-to-address translation to work properly, a unique name on the local network is necessary.
- In OS X, users can set a host name for their computers via the Local Hostname setting in the sharing pane of System Preferences.

Bonjour Solutions – Service Discovery

- Service discovery allows applications to find all available instances of a particular type of service and to maintain a list of named services and port numbers.
- The application can then resolve the service hostname to a list of IPv6 addresses as described in Naming.
- Service discovery is accomplished by “browsing” which is very much different from the traditional device-centric idea of network services.
- An mDNS query is sent out for a given service type and domain. Any matching services reply with their names. The result is a list of available services to choose.

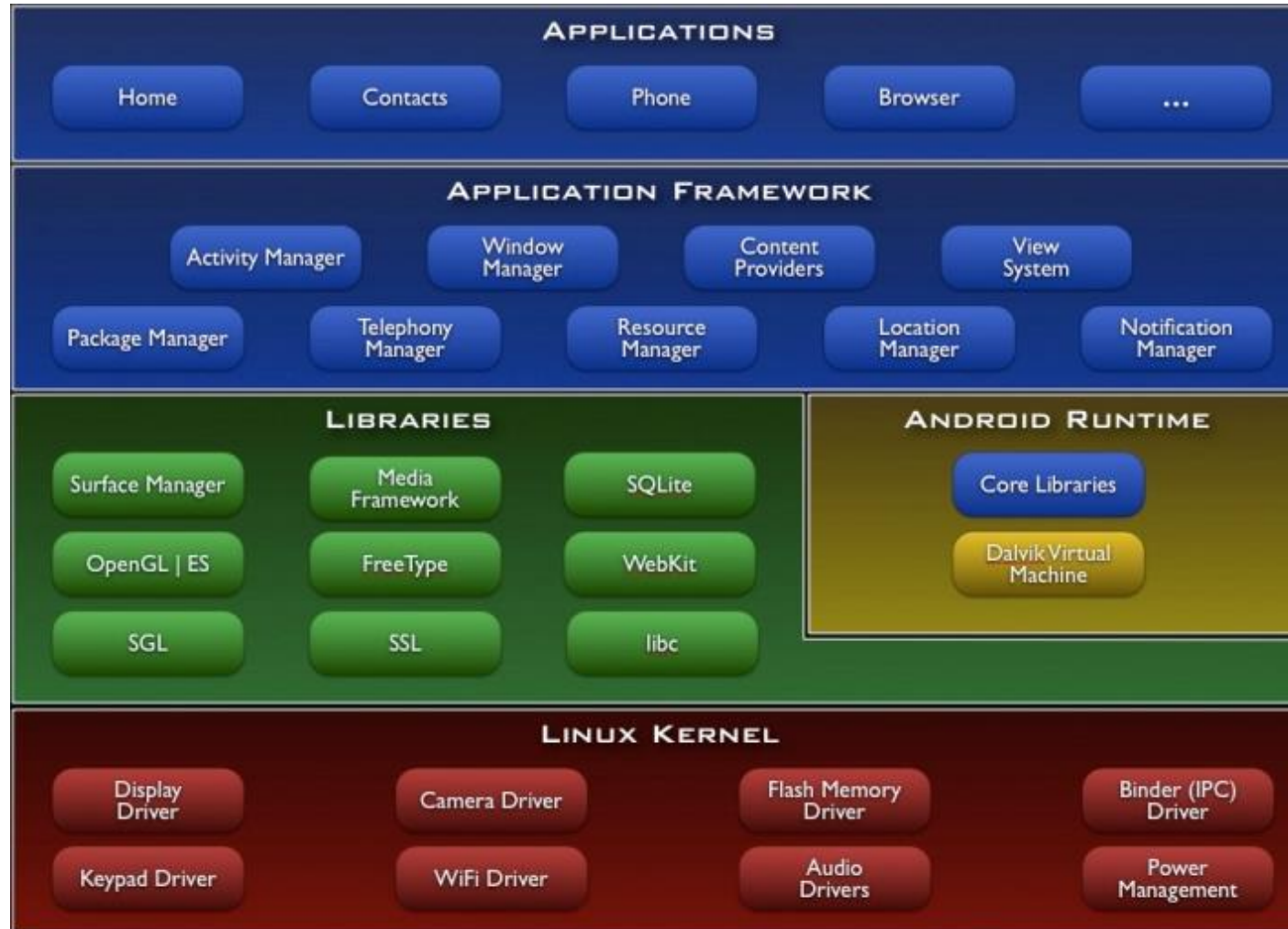
Bonjour Summarised

- In a nut shell, Bonjour consists of two parts
 - Service Registration
 - Service Discovery

Bonjour Across iOS & Android

- A cross platform service implements the same protocol on both iOS and Android
- Apps on both platforms require to advertise themselves as the same Bonjour service on the network, and respond with same kind of information
- Technically, Bonjour is for Apple iOS devices
- The Android implementations calls itself as Network Service Discovery
- Both are implementations of zero-configuration networking – DNS-SD and mDNS. In general, Bonjour.

Android OS Architecture



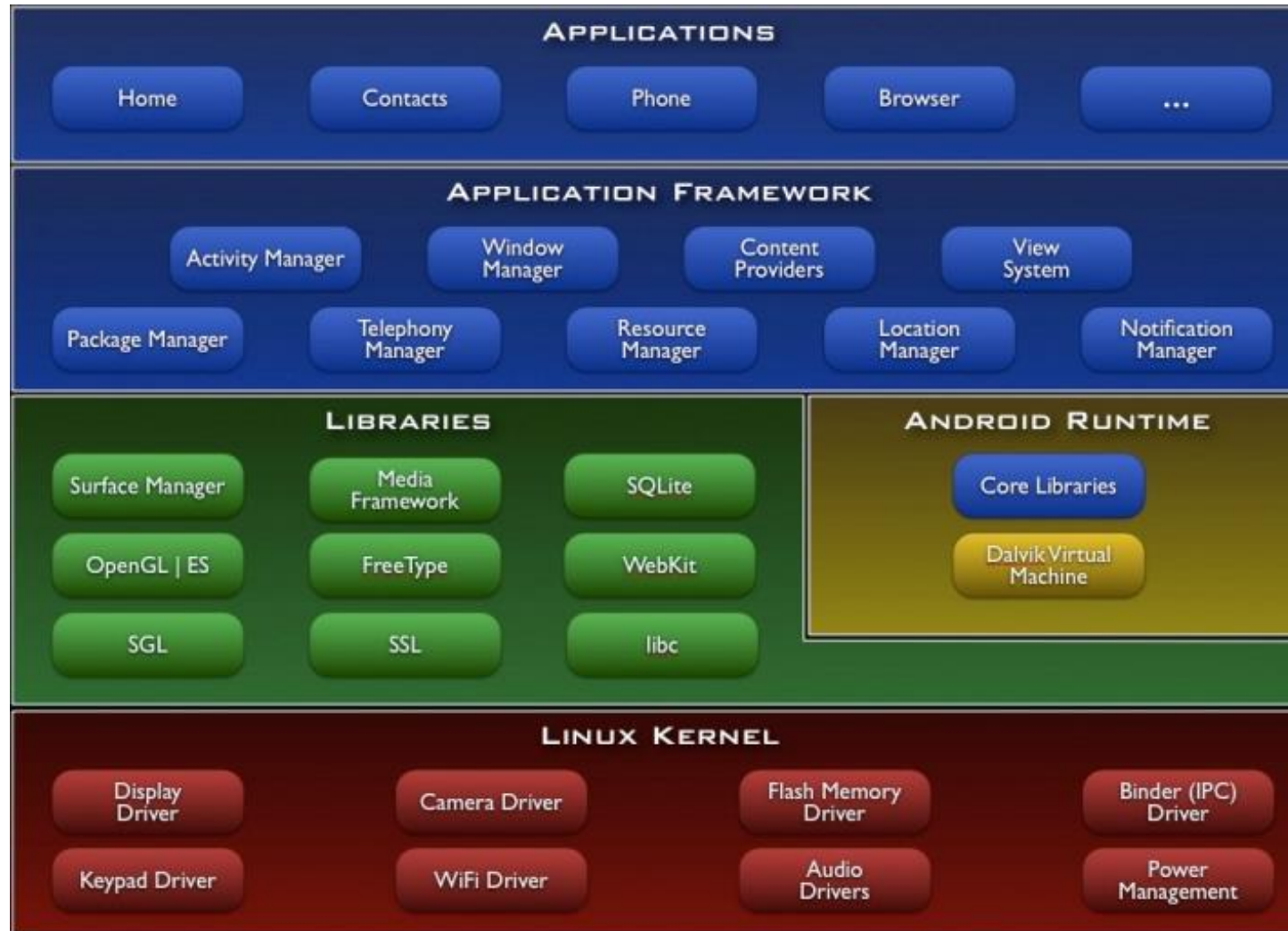
History of Android

- Android, originally meaning “robot”, is a mobile operating system using a modified version of the Linux kernel.
- It was initially developed by Android Inc., a firm later purchased by Google, and lately by the Open Handset Alliance (OHA).
- OHA is a consortium of 84 companies that includes Google, Samsung, Sony, HTC, Motorola, etc. to develop open standards for mobile devices
- It allows developers to write managed code in the Java language, controlling the device via Google-developed Java libraries.

Android at a Glance

- Software stack for mobile devices
 - incl. OS, middleware and key applications
- Open source – source code is open and contributions are welcome
- Application framework enabling reuse/replacement of apps
- Dalvik virtual machine optimized for mobile
- Integrated browser based on webkit
- Optimized graphics – 2D graphics library, 3D based on OPEN GL ES
- SQLite for data storage
- Media support (MPEG4, H.264, MP3, AAC, JPG, PNG,...)
- Support for radio interfaces, Bluetooth, WiFi, Camera, GPS, accelerometer
- Software Development Kit (SDK)
- Preinstalled applications from Google (GMS)
 - Gmail, Maps, Search, Voice Search, YouTube

Android Architecture



Android Architecture

- **Linux Kernel**

Android relies on Linux (Kernel) version 2.6 for core system services such as security, memory management, process management, network stack, and driver model. The kernel also acts as an abstraction layer between the hardware and the rest of the software stack.

Linux (Kernel) version 2.6 was made available during December 2003.

New releases are available for every 2 to 3 months time.

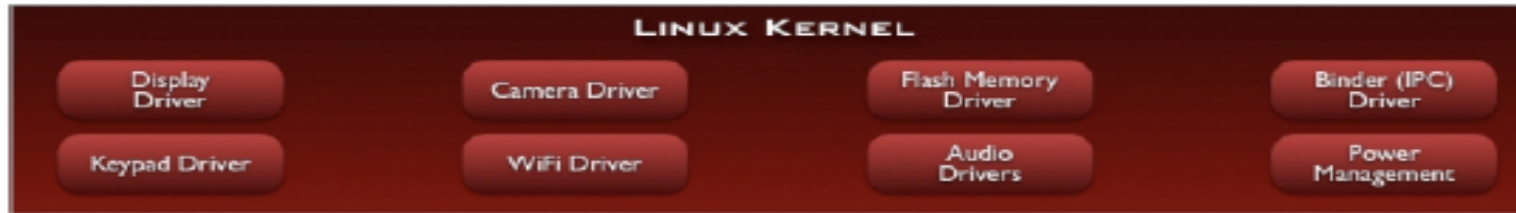
Android Architecture – Linux Kernel



Provides the following services to the upper layers:

- hardware abstraction – via registration of device
- memory management – buddy, slabs, pmem, etc.
- security settings – SE for android, tomoyo, smack
- power management – by /sys/power/state (vfs)
android aims to put system to sleep, apps can prevent this with a WAKE_LOCK
- device drivers – camera, bluetooth, wi-fi, audio, video
- file system support – ext4, fuse, fat, ntfs, ...
- network stack – socket buffers, protocols, etc.

Android Architecture – Linux Kernel



- **binder** – for interprocess communication (IPC)
a remote method invocation system

One process can call a function in another process and get a return object – ex: Window Manager ↔ client

A process may get an object from one object and pass it on to another object

- **logger** – kernel msgs are saved to persistent memory
- **ashmem** – shared library support

Android Architecture – Linux Kernel



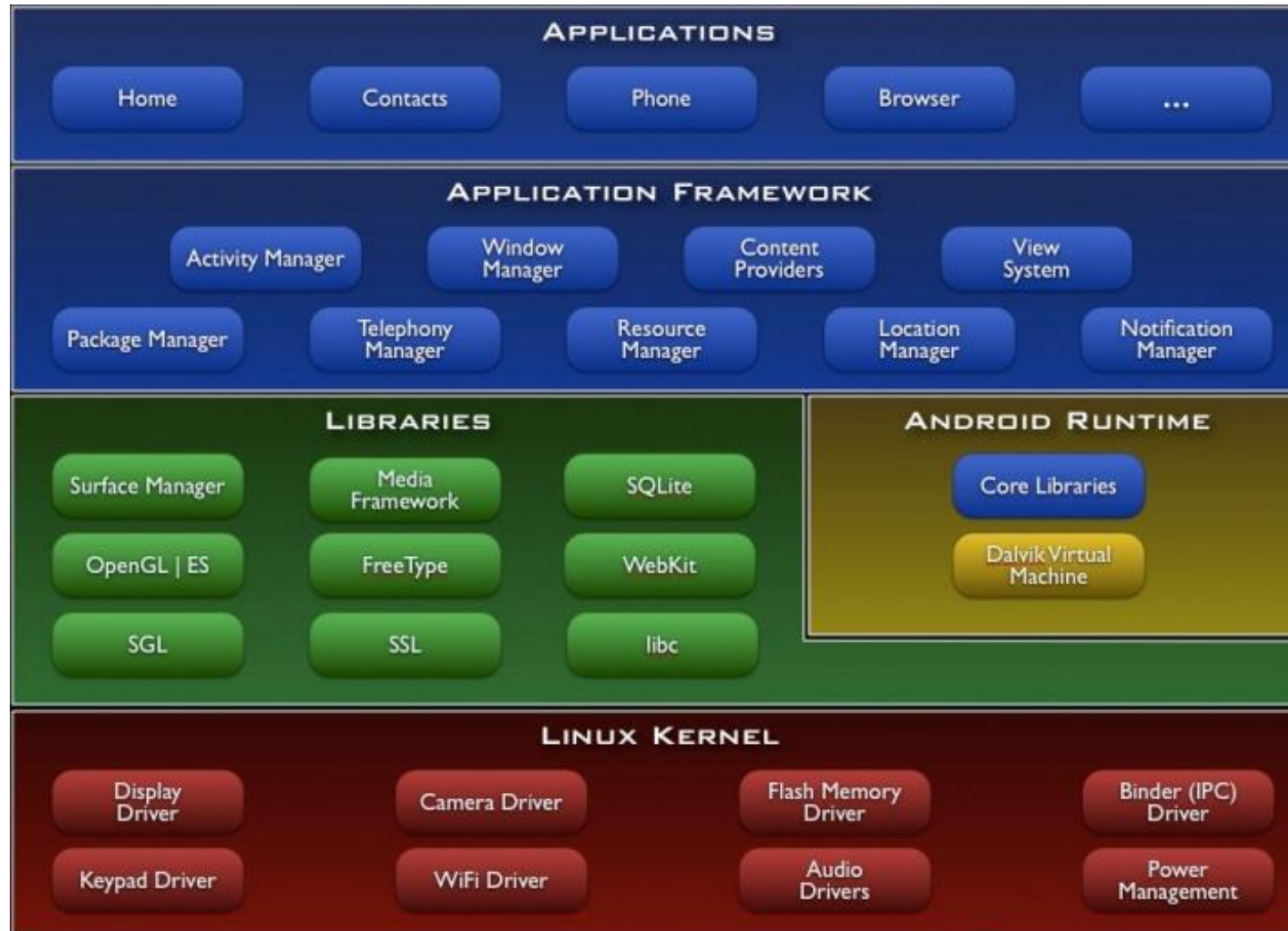
- Memory Management
 - Buddy & Slabs – memory management algorithms
 - pmem – Persistent memory – solid state, high performance, byte addressable memory
- Security
 - SE for Android – Security Enhance for Android by Samsung (Prevents apps or processing in accessing data or resources that are not permitted)
 - Tomoyo – for linux is mandatory access control (MAC) to increase security of the system
 - Smack – Simplified mandatory access control kernel – linux kernel security module – protects data and process interaction from malicious manipulation using mac rules.

Android Architecture – Linux Kernel



- File System Support
 - Ext4 – Fourth extended file system is a journaling file system for linux and successor of ext3
 - Fuse – File system in user space – software interface for linux and unix for creating own file system without editing kernel code
 - FAT – File Allocation Table – Originally designed for floppy disk use in 1977 in DOS and Windows
 - NTFS – New Technology File System – originally designed for Windows NT operating system

Android Architecture



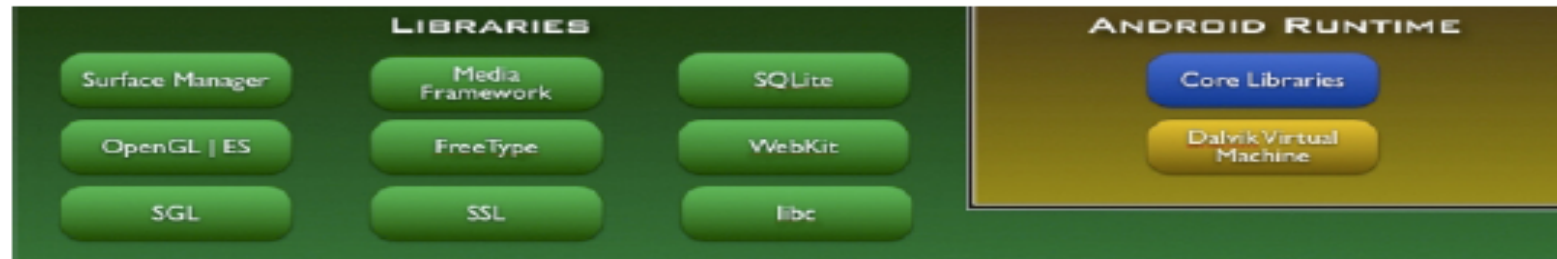
Android Architecture - Libraries

- Android includes a set of C/C++ libraries used by various components of the Android system. These capabilities are exposed to developers through the Android application framework. Some of the core libraries are listed below:
- **System C library** - a BSD-derived implementation of the standard C system library (libc), tuned for embedded Linux-based devices
- **Media Libraries** - based on PacketVideo's OpenCORE; the libraries support playback and recording of many popular audio and video formats, as well as static image files, including MPEG4, H.264, MP3, AAC, AMR, JPG, and PNG
- **Surface Manager** - manages access to the display subsystem and seamlessly composites 2D and 3D graphic layers from multiple applications

Android Architecture – Libraries (Contd...)

- **LibWebCore** - a modern web browser engine which powers both the Android browser and an embeddable web view
- **SGL** - the underlying 2D graphics engine
- **3D libraries** - an implementation based on OpenGL ES 1.0 APIs; the libraries use either hardware 3D acceleration (where available) or the included, highly optimized 3D software rasterizer
- **FreeType** - bitmap and vector font rendering
- **SQLite** - a powerful and lightweight relational database engine available to all applications

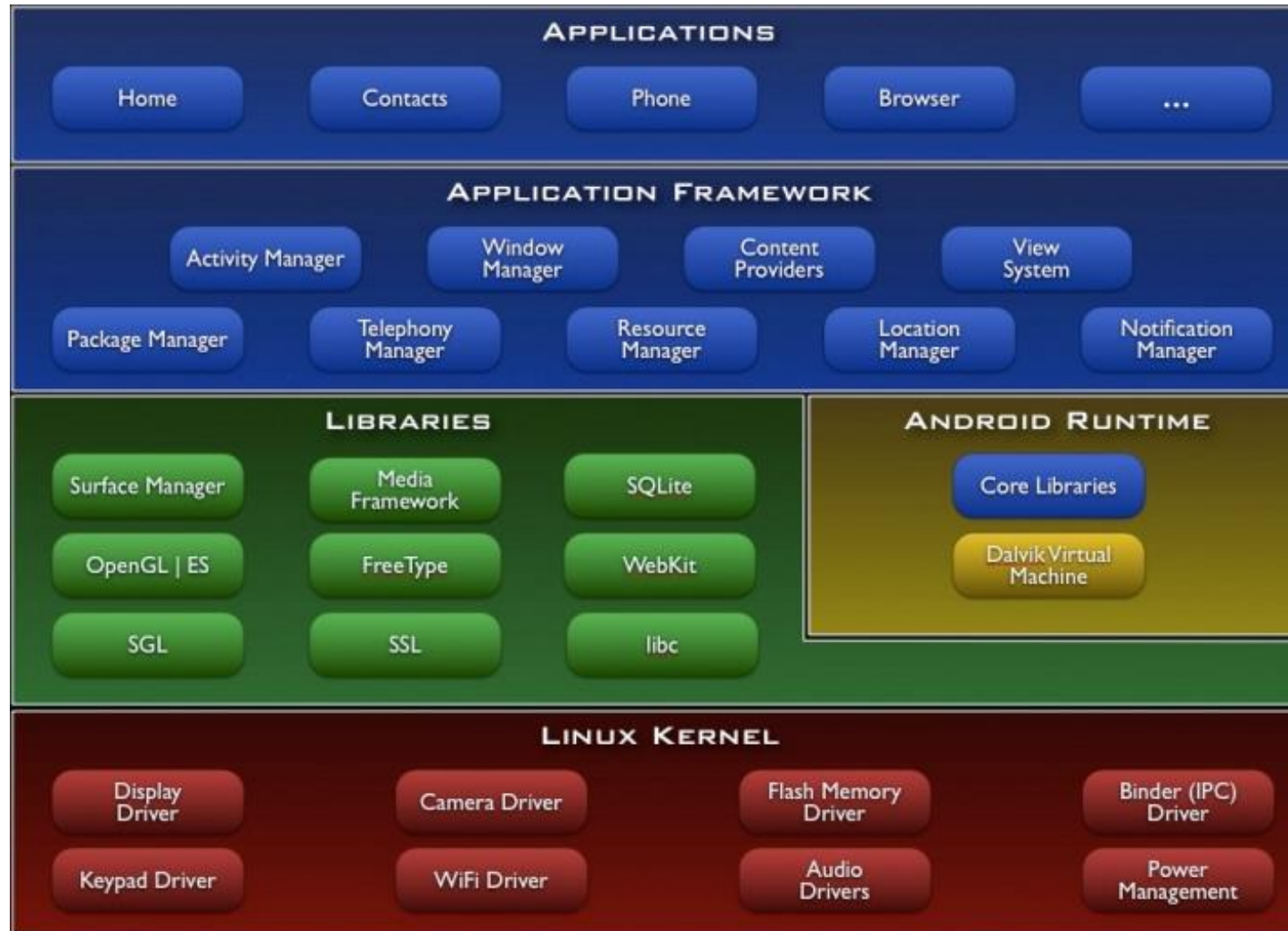
Android Architecture – Libraries (Contd...)



Native libraries for handling different kinds of data:

- surface manager – composes windows on the screen
- SGL – 2D graphics
- OpenGL – 3D graphics
- media framework – codecs for playback/recording of audio/video formats
- freetype – fonts and font rendering
- WebKit – browser engine
- SQLite – database management
- OpenSSL – secure communication, certificates, etc.

Android Architecture

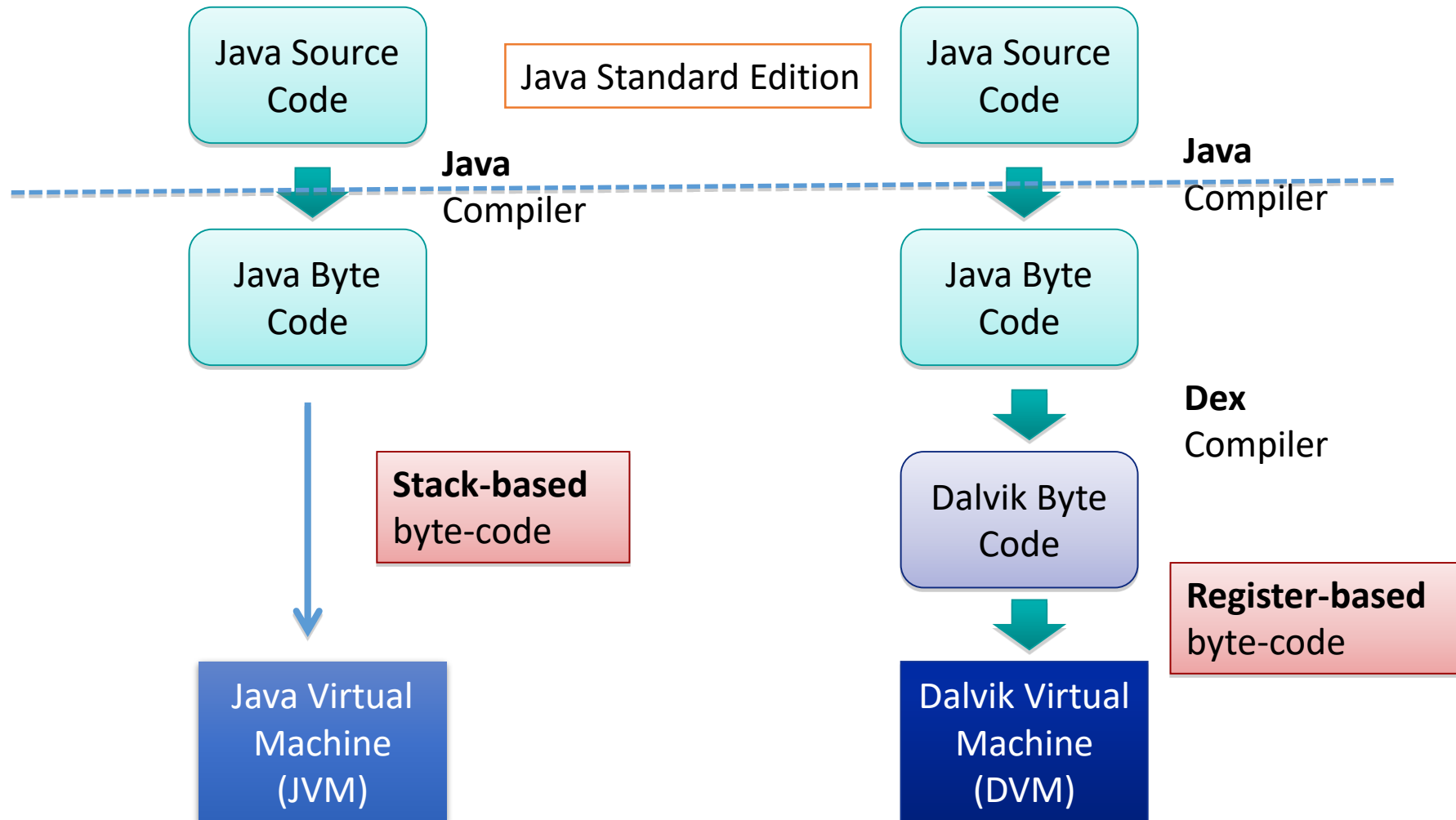


Android Architecture – Run Time

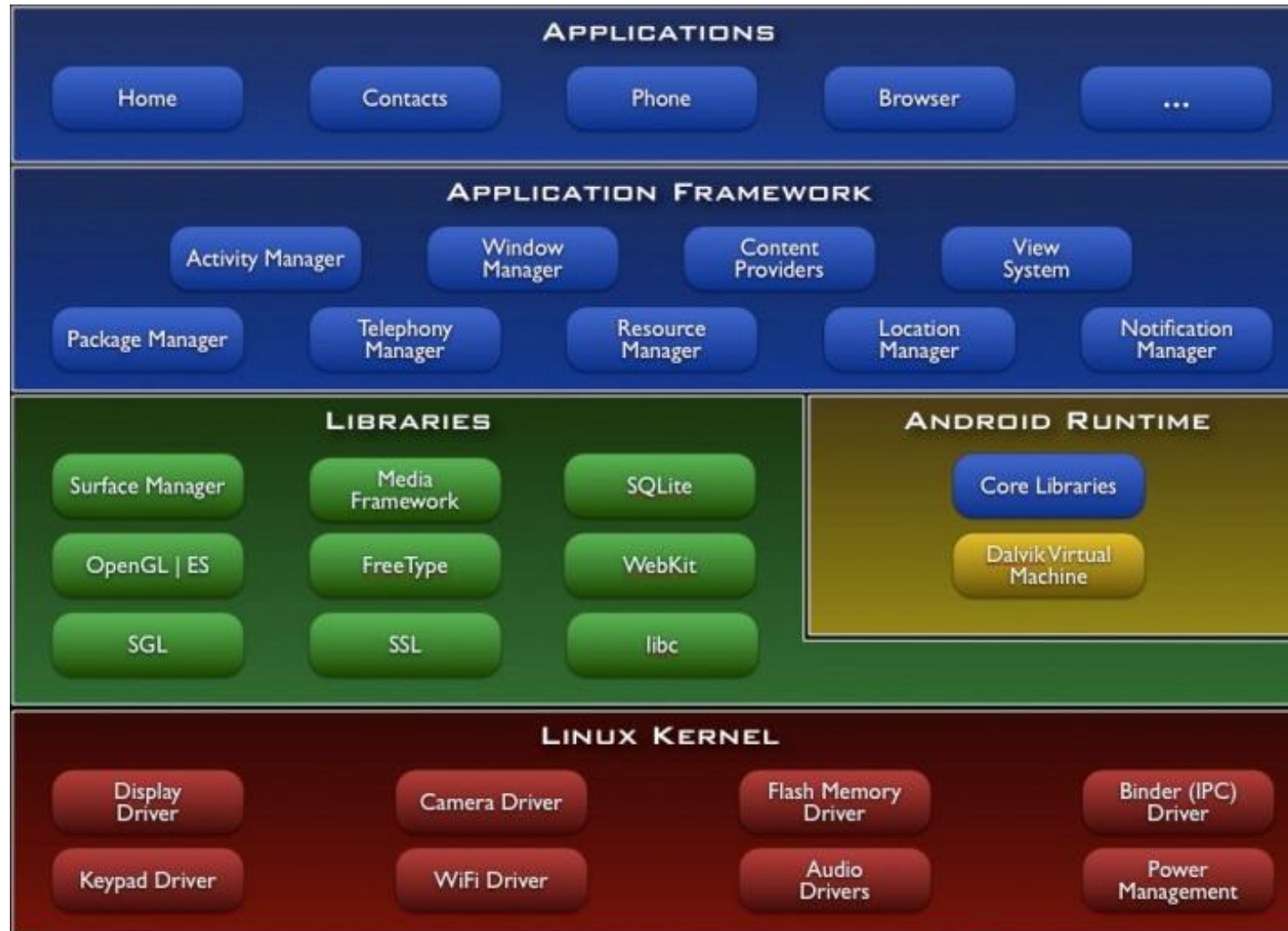


- Many embedded operating systems use Java Virtual Machine (JVM). But Android uses Dalvik Virtual Machine (DVM)
- DVM supports low configuration CPU, limited memory and most importantly limited battery power
- Java class files run using JVM but Java programs for Android are translated into “dex” files run on DVM
- Dalvik can run multiple VMs efficiently
- DVM uses its own byte code not Java byte code
- Android includes a set of core libraries that provides most of the functionality available in the core libraries of the Java programming language

Dalvik/Java Virtual Machines



Android Architecture



Android Architecture – Application Framework

- Developers are free to take advantage of the device hardware, access location information, run background services, set alarms, add notifications to the status bar, and much, much more.
- Developers have full access to the same framework APIs used by the core applications. The application architecture is designed to simplify the reuse of components; any application can publish its capabilities and any other application may then make use of those capabilities (subject to security constraints enforced by the framework). This same mechanism allows components to be replaced by the user.

Android Architecture – Application Framework (Contd...)

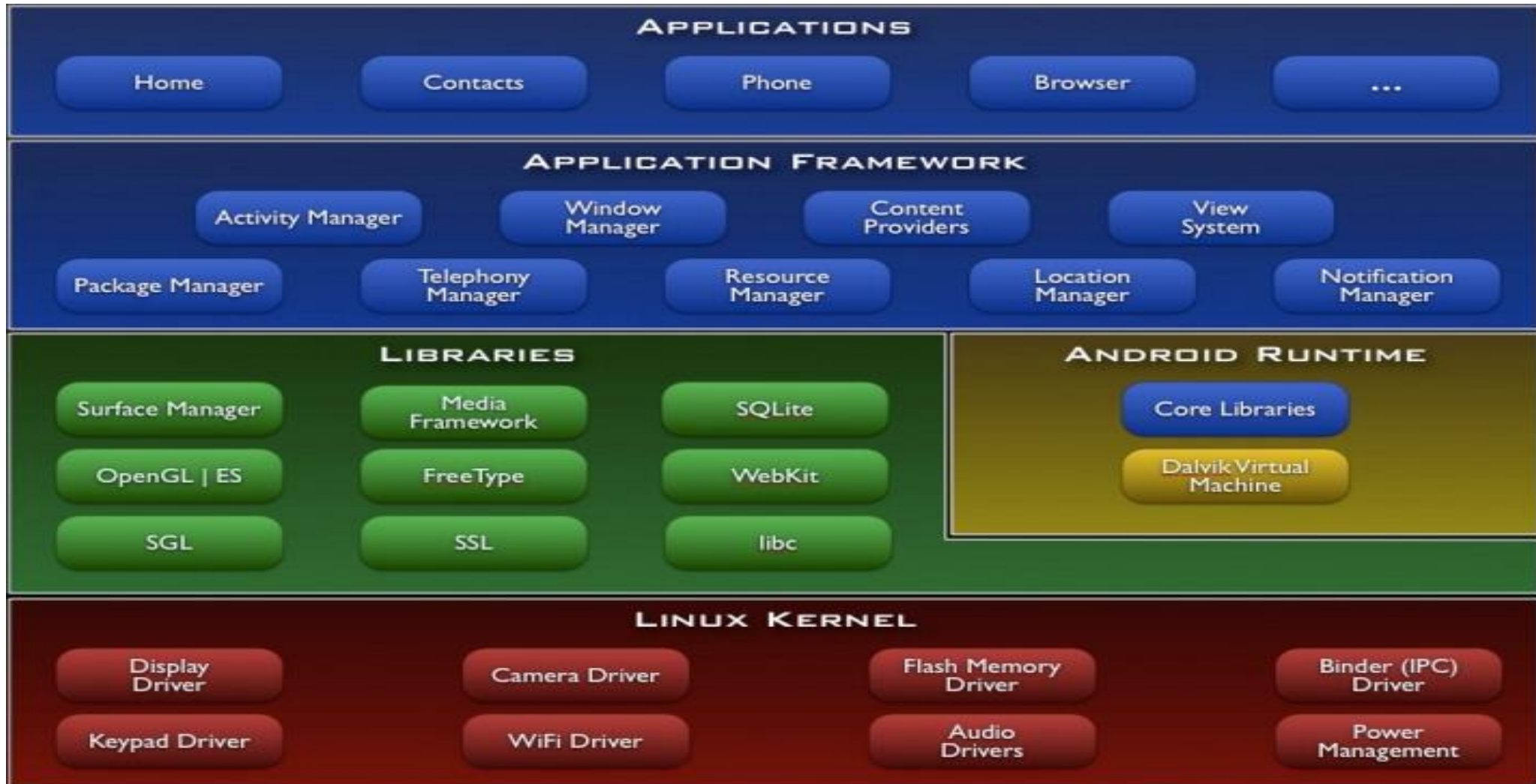


Manage basic phone functions:

(see `bin/frameworks/base/services/java/com/android/server`)

- **Activity Manager** – provides screen(s) in which user interacts with app (each is an activity)
- **Content Providers** – manage data sharing between apps
- **Telephony Manager** – manage voice calls
- **Location Manager** – GPS or cell tower locator
- **Resource Manager** – manage resources used by apps (memory, files, devices, locks, etc.)

Android Architecture - Applications



Android Architecture – Applications (Contd...)



- Android applications are written in the Java programming language.
- The compiled Java code — along with any data and resource files required by the application — is bundled by the [aapt tool](#) into an *Android package*, an archive file marked by an .apk suffix.
- aapt – Android Asset Packaging Tool is part of SDK and allows to view create and update “apk” file (apk is a zip compatible file).
- This file is the vehicle for distributing the application and installing it on mobile devices; it's the file users download to their devices. All the code in a single .apk file is considered to be one *application*.

Application Fundamentals

- By default, every application runs in its own Linux process. Android starts the process when any of the application's code needs to be executed, and shuts down the process when it's no longer needed and system resources are required by other applications.
- Each process has its own virtual machine (VM), so application code runs in isolation from the code of all other applications.

Application Components

- An Android application can make use of elements of other applications. If your application needs to display a scrolling list of images and another application has developed a suitable scroller and made it available to others, you can call upon that scroller to do the work.
- Your application simply starts up that piece of the other application when the need arises.
- The system must start an application process when any part of it is needed, and instantiate the Java objects for that part. Android applications don't have a single entry point (no `main()` function).
- Android applications have essential *components* that the system can instantiate and run as needed.

Android Application Main Components

- Activities
- Services
- Content Providers
- Broadcast Receivers

Application Components - Activities

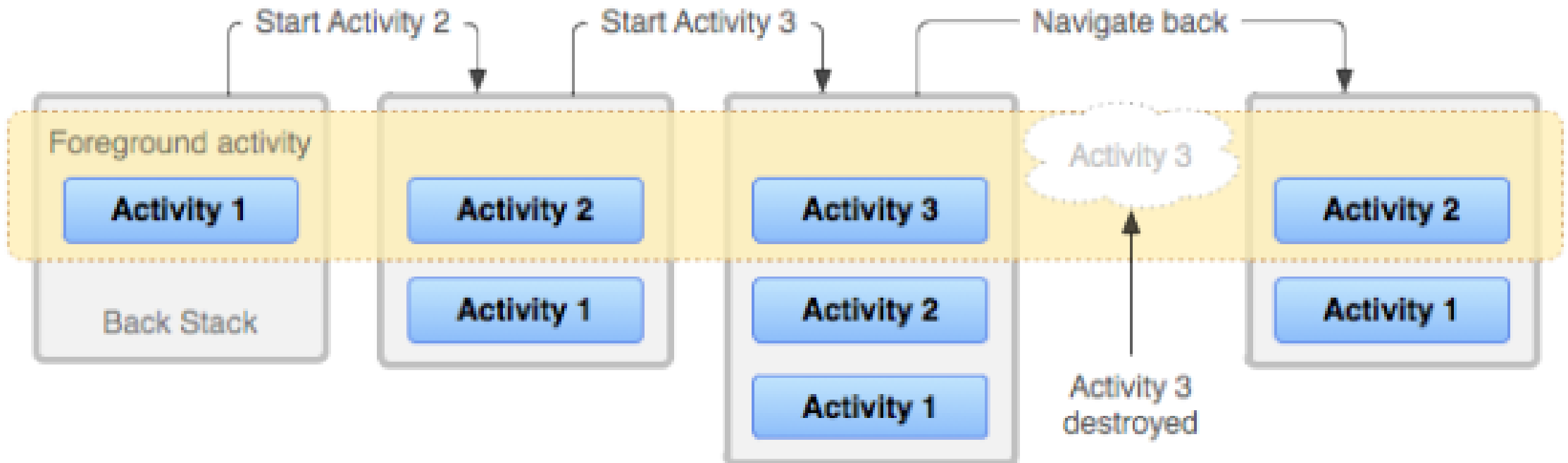
- An Activity represents a single screen
- An Activity represents the visual user interface for one focused endeavor the user can undertake.
- Example - Present a list of menu items users can choose from or it might display photographs along with their captions.
- Example - A text messaging application might have one activity that shows a list of contacts to send messages to, a second activity to write the message to the chosen contact, and other activities to review old messages or change settings.
- Each activity is independent of the others.
- Each one is implemented as a subclass of the [Activity](#) base class.

Application Components – Activities (Contd...)

```
public class MainActivity extends Activity {  
}
```

- Typically, one of the activities is marked as the first one that should be presented to the user when the application is launched.
- Moving from one activity to another is accomplished by having the current activity start the next one.
- Each time a new activity is started, the previous activity will be stopped and the system will save it in a back stack.
- Return to the previous activity by clicking back button, the system will grab it from the back stack.

Application Components – Activities (Contd...)



Application Components – Activities (Contd...)

- The visual content of the window is provided by a hierarchy of views — objects derived from the base [View](#) class.
- Each view controls a particular rectangular space within the window. Parent views contain and organize the layout of their children.
- Leaf views (those at the bottom of the hierarchy) draw in the rectangles they control and respond to user actions directed at that space.

Application Components – Activities (Contd...)

- Views are where the activity's interaction with the user takes place. A view might display a small image and respond when the user taps that image.
- Android has a number of ready-made views that you can use — including buttons, text fields, scroll bars, menu items, check boxes, and more.
- A view hierarchy is placed within an activity's window by the [Activity.setContentView\(\)](#) method.
- The *content view* is the View object at the root of the hierarchy.

Application Components – Services

- A *service* doesn't have a visual user interface, but rather runs in the background for an indefinite period of time.
- Each service extends the [Service](#) base class.
- A good example of a service is a media player playing songs. One or more player activities would allow the user to choose songs and start playing them. The music playback itself would not be handled by an activity (users will expect the music to keep playing). The media player activity could start a service to run in the background.

```
public class MyService extends Service {  
  
}
```

Application Components – Services (Contd...)

- It's possible to connect to (bind to) an ongoing service (and start the service if it's not already running). While connected, you can communicate with the service through an interface that the service exposes. For the music service, this interface might allow users to pause, rewind, stop, and restart the playback.
- Like activities and the other components, services run in the main thread of the application process. So that they won't block other components or the user interface, they often spawn another thread for time-consuming tasks (like music playback).
- Another application component such as an Activity could start a service so that it runs independently or linked to interact with it.

Application Components – Services (Contd...)

- Services are basically two types:
 - **Started**
 - When booting up, the service will operate in background indefinitely, even after its launching components ended.
 - Usually a “started service” only be assigned to a single processor and not returning to the component called it.
 - **Bound**
 - A bound service provides a client-server interface to allow components can interact with each other.
 - Life time of bound service component depends on links (bound) with it. Which means, when the component link exists, its bound service exists or vice versa.
 - Many components can be linked with a bound service at a time. But when all these components are unbounded then this type of service is cancelled.

Application Components – Broadcast Receivers

- A *broadcast receiver* is a component that does nothing but receive and react to broadcast announcements.
- Many broadcasts originate in system code — for example, announcements that the time zone has changed, that the battery is low, that a picture has been taken, or that the user changed a language preference.
- Applications can also initiate broadcasts — for example, to let other applications know that some data has been downloaded to the device and is available for them to use.
- An application can have any number of broadcast receivers to respond to any announcements it considers important. All receivers extend the [BroadcastReceiver](#) base class.

Application Components – Broadcast Receivers (Contd...)

```
public class MyReceiver extends BroadcastReceiver {  
    public void onReceive(context,intent){  
    }  
}
```

- Broadcast receivers do not display a user interface. However, they may start an activity in response to the information they receive, or they may use the [NotificationManager](#) to alert the user.
- Notifications can get the user's attention in various ways — flashing the backlight, vibrating the device, playing a sound, and so on. They typically place a persistent icon in the status bar, which users can open to get the message.

Application Components – Content Providers

- A *content provider* makes a specific set of the application's data available to other applications. The data can be stored in the file system, in an SQLite database, or in any other manner that makes sense.
- Content provider manages a set of shared data applications. For example, OS provides a Content Management Provider phonebook. An application with proper permission can read or write information into the phonebook.
- The content provider extends the [ContentProvider](#) base class to implement a standard set of methods that enable other applications to retrieve and store data of the type it controls.

Application Components – Content Providers

- The content provider extends the [ContentProvider](#) base class to implement a standard set of methods that enable other applications to retrieve and store data of the type it controls.
- Applications do not call these methods directly. Rather they use a [ContentResolver](#) object and call its methods instead. A ContentResolver can talk to any content provider; it cooperates with the provider to manage any interprocess communication that's involved.

```
public class MyContentProvider extends ContentProvider {  
    public void onCreate(){  
    }  
}
```

Android Application Additional Components

Components	Description
Fragments	Represents a portion of user interface in an Activity.
Views	UI elements that are drawn on-screen including buttons, lists forms etc.
Layouts	View hierarchies that control screen format and appearance of the views.
Intents	Messages wiring components together.
Resources	External elements, such as strings, constants and drawable pictures.
Manifest	Configuration file for the application.

Additional Components – Intent

- Intent is basically a message that is passed between components (such as Activities, Services, Broadcast Receivers, and Content Providers). So, it is almost equivalent to parameters passed to API calls. The fundamental differences between API calls and intents' way of invoking components are:
 - API calls are synchronous while intent-based invocations are asynchronous.
 - API calls are compile time binding while intent-based calls are run-time binding. Of course, Intents can be made to work exactly like API calls by using what are called explicit intents, which will be explained later. But more often than not, implicit intents are the way to go and that is what is explained here.

Additional Components – Intent (Contd...)

- One component that wants to invoke another has to only express its' intent to do a job. And any other component that exists and has claimed that it can do such a job through intent-filters, is invoked by the android platform to accomplish the job. This means, both the components are not aware of each other's existence and can still work together to give the desired result for the end-user.
- Three of the four types of main components – Activity, Service and Broadcast Receiver are launched by an asynchronous call, called Intent
- Content providers are activated when they're targeted by a request from a ContentResolver.
- Intent creates a linked basis between the components in the application together
- An intent is an [Intent](#) object that holds the content of the message.

Additional Components – Intent (Contd...)

- For activities and services, the Intent names the action being requested and specifies the URL of the data to act on. For example, it might convey a request for an activity to present an image to the user or let the user edit some text.
- For broadcast receivers, the Intent object names the action being announced. For example, it might announce to interested parties that the camera button has been pressed.
- This invisible connection between components is achieved through the combination of intents, intent-filters and the android platform.

Additional Components – Intent (Contd...)

- Methods for Activating an Activity
 - An activity is launched (or given something new to do) by passing an Intent object to [Context.startActivity\(\)](#) or [Activity.startActivityForResult\(\)](#).
 - The responding activity can look at the initial intent that caused it to be launched by calling its [getIntent\(\)](#) method.
 - Android calls the activity's [onNewIntent\(\)](#) method to pass it any subsequent intents.
 - One activity often starts the next one. If it expects a result back from the activity it's starting, it calls `startActivityForResult()` instead of `startActivity()`. For example, if it starts an activity that lets the user pick a photo, it might expect to be returned the chosen photo. The result is returned in an Intent object that's passed to the calling activity's [onActivityResult\(\)](#) method.

Additional Components – Intent (Contd...)

- Methods for Activating a Service
 - A service is started (or new instructions are given to an ongoing service) by passing an Intent object to [Context.startService\(\)](#).
 - Android calls the service's [onStart\(\)](#) method and passes it the Intent object.
 - Similarly, an intent can be passed to [Context.bindService\(\)](#) to establish an ongoing connection between the calling component and a target service. The service receives the Intent object in an [onBind\(\)](#) call. (If the service is not already running, [bindService\(\)](#) can optionally start it.)
 - For example, an activity might establish a connection with the music playback service mentioned earlier so that it can provide the user with the means (a user interface) for controlling the playback. The activity would call [bindService\(\)](#) to set up that connection, and then call methods defined by the service to affect the playback.
 - An application can initiate a broadcast by passing an Intent object to methods like [Context.sendBroadcast\(\)](#), [Context.sendOrderedBroadcast\(\)](#), and [Context.sendStickyBroadcast\(\)](#) in any of their variations.
 - Android delivers the intent to all interested broadcast receivers by calling their [onReceive\(\)](#) methods.

Additional Components – Intent (Contd...)

- Shutting down components
 - Content providers and broadcast receivers don't need to be explicitly shut down.
 - A content provider is active only while it's responding to a request from a ContentResolver.
 - A broadcast receiver is active only while it's responding to a broadcast message.
 - Activities, on the other hand, provide the user interface. They're in a long-running conversation with the user and may remain active, even when idle, as long as the conversation continues.
 - Services may also remain running for a long time. Android has methods to shut down activities and services in an orderly way.

Additional Components – Intent (Contd...)

- Shutting down components (Contd...)
 - An activity can be shut down by calling its [`finish\(\)`](#) method. One activity can shut down another activity (one it started with `startActivityForResult()`) by calling [`finishActivity\(\)`](#).
 - A service can be stopped by calling its [`stopSelf\(\)`](#) method, or by calling [`Context.stopService\(\)`](#).
 - Components might also be shut down by the system when they are no longer being used or when Android must reclaim memory for more active components.

Android Applications

- Client Apps
- Web Apps

Android Applications (Contd...)

- Client Apps

- Developed using the Android SDK and installed on user devices
- Compiled Java code, with data and resource – bundled by Android Asset Packaging tool (AAPT) into Android package or .apk
- All applications have Android Manifest file in its root directory
 - provides essential information about app
- Could be installed directly on phone, but necessary to be distributed thru Market

- Web Apps

Android Applications (Contd...)

- Client Apps
- Web Apps
 - An alternative to standalone apps
 - Developed using web standards and accessed through browser – nothing to install on devices
 - Mixing client and web apps is also possible – Client apps can embed web pages using “WebView” in Android app

THANK YOU

END OF
MOBILE OS ARCHITECTURE